

IoT 设备 RCE 漏洞挖掘通用技术文档（基于公开案例的二进制分析方法论）

适用对象：具有基础 C/C++、Linux、嵌入式系统、逆向工程知识的安全研究人员。

工具限制：本文以 IDA Pro 9、x64dbg、x32dbg 为核心分析工具，辅以少量系统工具（binwalk、strings、readelf 等）作为辅助信息获取工具。

文档目标：建立一套“从固件获取 → 文件系统分析 → 二进制逆向 → 危险函数识别 → 输入点定位 → 漏洞成因分析 → 漏洞验证 → 风险评估”的通用 IoT RCE 漏洞研究流程。

合规说明：本文仅用于授权环境下的漏洞研究、代码审计、防御验证与安全教学。所有案例均基于公开披露漏洞所总结的方法论，不提供面向真实目标的攻击自动化脚本、武器化利用链或未授权渗透指导。

第一章 IoT 漏洞研究的基本认知

1.1 为什么 IoT 设备容易出现 RCE

相比传统桌面软件，IoT 设备在安全工程层面存在以下典型问题：

- 开发周期短
- 固件生命周期长
- 供应链复杂
- 使用大量第三方 SDK
- BusyBox 与裁剪版 libc 广泛存在
- 安全编译选项缺失
- Web 管理接口代码质量低
- 厂商缺乏 SDL（Secure Development Lifecycle）

典型攻击面包括：

- Web CGI 接口
- UPnP 服务
- MQTT 服务
- Telnet/SSH 管理接口
- 自定义 TCP/UDP 服务
- SOAP 接口
- 固件升级接口
- 云通信组件
- IPC 通信模块

很多 IoT RCE 本质上都来自：

- 输入验证缺失
- 命令拼接

- 栈缓冲区溢出
- 堆溢出
- 格式化字符串
- 整数溢出
- 路径穿越导致配置覆盖
- 认证绕过调用危险接口

因此，IoT RCE 挖掘并不是单纯“找溢出”，而是寻找：

用户可控输入如何最终影响危险执行路径。

1.2 IoT 固件的典型结构

一个常见路由器固件可能包含：

- Linux Kernel
- BusyBox
- lighttpd/httpd/goahead
- CGI 程序
- 动态库 (.so)
- NVRAM 配置系统
- Web 前端资源
- watchdog 进程
- 厂商守护进程

典型目录：

```
/bin
/sbin
/usr/bin
/usr/sbin
/www
/etc
/lib
```

在漏洞研究中，重点关注：

- CGI 可执行文件
 - 网络守护进程
 - 处理 socket 输入的服务
 - 使用 system/popen 的组件
-

第二章 固件获取与文件系统分析

2.1 固件获取来源

常见来源：

- 厂商官网
- OTA 更新包
- GitHub 泄露 SDK
- ISP 固件镜像
- UART dump
- SPI Flash 提取

公开漏洞研究中，大量案例都来自：

- 厂商升级包分析
- binwalk 提取 squashfs

2.2 使用 binwalk 提取固件

虽然本文核心工具是 IDA 与 x64dbg/x32dbg，但在正式逆向前必须先解析固件。

示例：

```
binwalk -Me firmware.bin
```

常见文件系统：

- squashfs
- cramfs
- ubifs
- jffs2

提取后重点定位：

```
find . -type f | grep -E "cgi|httpd|goahead|boa"
```

以及：

```
find . -type f -executable
```

2.3 快速识别高风险组件

使用 strings:

```
strings httpd | less
```

重点观察:

- system
- popen
- snprintf
- strcpy
- nvram_get
- ping
- traceroute
- iptables
- wget
- curl

如果出现:

```
system(user_input)
```

则存在明显命令注入风险。

第三章 IDA Pro 9 在 IoT 漏洞研究中的核心 workflow

3.1 导入二进制

IoT 固件常见架构:

- MIPS
- ARM
- ARM64
- x86

导入时需要注意:

- 是否为 stripped binary
- 是否 PIE
- 是否静态链接

在 IDA 中重点确认:

- Processor Type
- Endianness

- Compiler ABI

MIPS little endian 与 big endian 混淆会导致伪代码错误。

3.2 建立函数语义

新手最容易犯的错误：

只看伪代码，不看汇编。

正确流程：

1. 查看函数交叉引用
2. 确认参数来源
3. 确认危险函数调用
4. 回溯输入来源
5. 分析长度检查
6. 确认是否存在边界缺陷

重点快捷键：

- X: 查看交叉引用
 - F5: 反编译
 - G: 跳转地址
 - N: 重命名
 - Y: 修改类型
-

3.3 构建危险函数索引

这是 IoT 漏洞挖掘效率最高的方法之一。

重点搜索：

```
strcpy
strcat
sprintf
vsprintf
gets
memcpy
system
popen
snprintf
strncpy
recv
read
fread
```

注意：

`snprintf` 不一定安全。

很多厂商代码：

```
snprintf(buf, 0x400, user_input);
```

这里 `user_input` 被作为 format string。

属于格式化字符串风险。

3.4 使用字符串反向定位漏洞

IoT 固件中大量危险逻辑都能通过字符串快速定位。

例如：

```
ping  
diagnostic  
setWan  
apply.cgi
```

可能对应：

- 网络诊断功能
- 配置写入
- shell 命令执行

例如：

```
sprintf(cmd, "ping %s", ip);  
system(cmd);
```

如果 `ip` 可控，则可能命令注入。

第四章 IoT RCE 的主要漏洞类型

4.1 命令注入（最常见）

成因

开发者将用户输入拼接到 shell 命令。

典型危险代码：

```
sprintf(cmd, "ping %s", user);  
system(cmd);
```

如果输入：

```
127.0.0.1;reboot
```

则可能导致额外命令执行。

在 IDA 中如何发现

搜索：

```
system  
popen  
execl  
execve
```

然后向上回溯参数来源。

关键问题：

- 参数是否来自 HTTP 请求
- 是否经过过滤
- 黑名单是否不完整

常见错误过滤

例如：

```
if(strchr(input, ';'))
```

但未过滤：

```
&&  
|  
,  
$()
```

4.2 栈缓冲区溢出

基本原理

固定长度栈缓冲区接收超长输入。

例如：

```
char buf[64];
strcpy(buf, user_input);
```

如果输入超过 64 字节：

- 覆盖返回地址
- 覆盖栈变量
- 导致崩溃

IoT 中的高危场景

常见于：

- CGI 参数解析
- 自定义协议解析
- HTTP Header 处理
- SOAP XML 解析

IDA 分析重点

观察：

```
char local_80[64]
```

再观察：

```
strcpy(local_80, input)
```

然后确认：

- 是否存在长度检查
- 长度检查是否错误

4.3 堆溢出

IoT 固件中经常出现：

```
malloc(user_len)
memcpy(heap, input, real_len)
```

如果：

```
real_len > user_len
```

则可能堆破坏。

堆漏洞研究难点：

- allocator 行为不同
- uClibc 与 glibc 差异
- 设备内存布局随机性较低

4.4 格式化字符串漏洞

典型错误：

```
printf(user_input);
```

风险：

- 信息泄露
- 任意写
- RCE

IoT 中经常出现于日志函数。

例如：

```
syslog(user_input);
```

4.5 整数溢出

典型：

```
len = user_len + 0x20;
```

如果发生整数回绕：

```
malloc(len)
memcpy(...)
```

可能导致越界。

第五章 输入点定位方法论

5.1 HTTP CGI 分析

很多 IoT Web 服务结构：

```
POST /apply.cgi
```

然后：

```
websGetVar()
```

获取参数。

重点分析：

```
websGetVar
get_cgi
getenv
cgiFormString
```

例如：

```
v = websGetVar(wp, "ip", "");
```

继续追踪 v。

5.2 socket recv 输入

重点函数：

```
recv
read
fgets
```

例如：

```
recv(sock, buf, 1024, 0);
```

之后查看：

- 是否 strcpy
- 是否 sscanf
- 是否 strcat

5.3 NVRAM 配置注入

大量 IoT 固件使用：

```
nvrn_get  
nvrn_set
```

如果配置值最终进入：

```
system()
```

则可能形成持久化 RCE。

第六章 从公开漏洞案例中总结通用模式

6.1 路由器诊断功能命令注入

大量公开案例中：

```
ping diagnostics  
traceroute diagnostics
```

都是高危点。

原因：

开发者需要调用系统命令。

常见代码结构：

```
sprintf(cmd, "ping -c 4 %s", host);  
system(cmd);
```

研究重点：

- host 是否过滤
- 是否 URL decode
- 是否双重编码

6.2 SOAP 服务栈溢出

很多旧设备使用 SOAP。

风险：

- XML parser 不安全
- strcpy 解析节点值

研究方法：

1. 找 SOAP action
2. 定位 handler
3. 分析 XML value 长度处理

6.3 UPnP 解析漏洞

UPnP 经常存在：

- strcpy
- sscanf
- XML parsing overflow

重点：

```
sscanf(user, "%s", buf)
```

没有宽度限制。

第七章 使用 IDA 构建数据流分析思维

7.1 不要只看危险函数

真正核心：

用户输入是否可控到危险点。

例如：

```
recv -> parse -> strcpy
```

需要完整追踪。

7.2 建立 taint 思维

将用户输入视为 tainted data。

分析：

```
source -> transform -> sink
```

Source：

- recv
- CGI
- socket

Sink：

- system
 - strcpy
 - memcpy
-

7.3 伪代码误导问题

IDA 伪代码有时会错误推断。

必须交叉验证：

- 汇编
- 栈布局
- 参数寄存器

尤其 MIPS。

第八章 动态调试方法

8.1 为什么 IoT 动态调试困难

问题：

- 非 x86 架构
- 无调试符号
- 无 shell
- watchdog 重启

因此很多研究者：

- QEMU 模拟
- chroot 固件
- 本地复现逻辑

8.2 x64dbg/x32dbg 的使用定位

虽然很多 IoT 设备是 ARM/MIPS，但仍有部分：

- x86 NAS
- IPC 软件
- Windows 管理组件

适合使用 x64dbg。

重点功能：

- 内存断点
- API 断点
- SEH 分析
- 栈回溯

8.3 动态验证缓冲区溢出

例如：

```
strcpy(buf, input)
```

在 strcpy 下断点。

观察：

- 输入长度

- 栈变化
- 返回地址是否被覆盖

重点寄存器：

ESP/RSP
EBP/RBP
EIP/RIP

8.4 利用崩溃判断可控性

如果崩溃地址：

41414141

说明：

返回地址可能可控。

但：

崩溃 $\neq$ 可利用。

还需分析：

- DEP
- ASLR
- NX
- PIE

第九章 IoT 漏洞利用缓解机制分析

9.1 NX

作用：

- 栈不可执行

研究重点：

是否需要 ROP。

9.2 ASLR

很多旧 IoT 设备：

- ASLR 弱
- 固定地址

导致利用难度下降。

9.3 Stack Canary

检查栈破坏。

在 IDA 中可观察：

```
__stack_chk_fail
```

9.4 RELRO

影响 GOT 覆写。

使用：

```
checksec
```

分析。

第十章 常见危险代码模式总结

10.1 strcpy

```
strcpy(dst, src)
```

高危原因：

无边界检查。

10.2 sprintf

```
sprintf(buf, "%s", input)
```

可能溢出。

10.3 strcat

```
strcat(buf, input)
```

容易累计越界。

10.4 system

```
system(user)
```

命令执行风险。

10.5 popen

```
popen(cmd, "r")
```

同样危险。

第十一章 漏洞验证思路

11.1 不要直接攻击真实设备

推荐：

- QEMU 仿真
 - 本地实验环境
 - 离线固件分析
-

11.2 崩溃验证

验证：

- 是否 crash
 - 是否 controllable
 - 是否 stable
-

11.3 命令注入验证

验证原则：

不要执行破坏性命令。

例如使用：

```
echo test
```

类低风险验证。

第十二章 新手最容易犯的错误

12.1 只会搜 strcpy

危险点很多。

包括：

- sscanf
 - memcpy
 - snprintf
 - custom parser
-

12.2 不理解调用链

真正重点：

输入如何流向危险点。

12.3 不理解架构差异

MIPS:

- delay slot
- 参数寄存器不同

ARM:

- Thumb
 - ARM mode
-

第十三章 老手进阶：从单点漏洞到攻击面建模

13.1 不再只找单漏洞

高级研究者会建立：

- attack surface map
 - trust boundary
 - privilege boundary
-

13.2 组件关系分析

例如：

```
web -> ipc -> daemon -> shell
```

真正危险点可能在 daemon。

13.3 Patch Diff 分析

高级技巧：

对比：

- 漏洞版本
- 修复版本

观察：

- 新增长度检查
- 参数过滤
- 删除危险调用

这是发现同类漏洞的重要方法。

第十四章 IoT 漏洞研究中的逆向技巧

14.1 识别自定义封装函数

很多厂商封装：

```
doSystem()
```

内部调用：

```
system()
```

因此不能只搜 system。

14.2 恢复结构体

IDA 中恢复结构体能显著提升理解。

重点：

- socket context
 - request object
 - session structure
-

14.3 识别状态机

很多协议：

- 多阶段解析
- 状态切换

漏洞可能在：

状态不一致。

第十五章 IoT 固件中常见第三方组件风险

15.1 BusyBox

历史上存在多类解析漏洞。

15.2 mini_httpd / boa / goahead

大量历史 RCE。

重点：

- CGI
 - path handling
 - auth bypass
-

15.3 OpenSSL 老版本

可能存在：

- overflow
 - parsing bug
-

第十六章 漏洞评级与影响分析

16.1 是否需要认证

区别巨大：

- pre-auth RCE
 - post-auth RCE
-

16.2 是否远程可达

LAN only 与 WAN exposure 风险不同。

16.3 是否稳定利用

很多 crash：

不可稳定利用。

第十七章 研究 workflow 总结

推荐流程：

```
graph TD; A[固件获取] --> B[文件系统提取]; B --> C[识别网络服务]; C --> D[字符串分析]; D --> E[危险函数定位]; E --> F[输入点追踪]; F --> G[数据流分析]; G --> H[漏洞假设]; H --> I[动态验证]; I --> J[风险评估];
```

固件获取
↓
文件系统提取
↓
识别网络服务
↓
字符串分析
↓
危险函数定位
↓
输入点追踪
↓
数据流分析
↓
漏洞假设
↓
动态验证
↓
风险评估

第十八章 一套高效的 IoT RCE 审计 Checklist

Web CGI

检查：

- websGetVar
- system
- sprintf
- strcpy

Socket Service

检查：

- recv
- memcpy

- sscanf
-

IPC

检查：

- message parser
 - shared memory
-

第十九章 如何阅读公开漏洞文章而不是机械复现

很多研究者误区：

只复制 payload。

真正应该学习：

- 漏洞成因
- 数据流
- trust boundary
- patch logic

例如：

厂商修复：

```
strcpy -> strncpy
```

不一定真正安全。

仍可能：

- truncation bug
 - missing NULL termination
-

第二十章 从研究者视角建立长期能力

20.1 建立漏洞模式库

记录：

- 命令注入模式
- parser overflow

- auth bypass

20.2 建立函数危险等级

例如：

函数	风险
system	极高
strcpy	极高
snprintf	中
memcpy	高

20.3 建立自己的 IDA 标记体系

例如：

- 红色：危险 sink
- 蓝色：用户输入
- 黄色：边界检查

第二十一章 一个完整的审计思维示例

假设分析一个 httpd。

步骤：

1. strings 查找 CGI
2. 找 apply.cgi handler
3. 找 websGetVar
4. 跟踪参数
5. 发现：

```
sprintf(cmd, "iptables -A %s", user)
```

1. 检查过滤
2. 判断是否命令注入
3. 验证是否可控
4. 分析权限

这是典型 IoT RCE 审计路径。

第二十二章 关于 exploitability 的现实认知

很多漏洞：

- 可以 crash
- 不能 RCE

真正 exploitability 受：

- heap layout
- mitigations
- architecture
- watchdog

影响。

因此：

不要把所有 overflow 都当成可利用漏洞。

第二十三章 高级方向：面向固件的大规模审计

高级研究会使用：

- 批量字符串扫描
- 固件聚类
- diff analysis
- 自动化 sink 识别

但核心能力仍然是：

手工理解数据流。

第二十四章 新手训练路线建议

阶段一：

- 学 C 内存模型
- 学栈结构
- 学 Linux 用户态

阶段二：

- 学 IDA
- 学汇编
- 学 calling convention

阶段三：

- 分析公开 IoT 漏洞
- 做 patch diff

阶段四：

- 自主审计固件

第二十五章 推荐研究方向

建议重点研究：

- 路由器 Web 管理接口
- NAS 服务
- IPC 摄像头
- 智能家居网关

因为：

- 攻击面广
- 历史漏洞多
- 二进制质量参差不齐

第二十六章 结语

IoT RCE 漏洞研究，本质上是：

对“不可信输入如何突破边界并影响危险执行路径”的系统化分析。

真正成熟的研究者，不会停留在：

- 搜 strcpy
- 跑 PoC
- 复制 payload

而是会建立：

- 数据流思维
- 边界分析能力
- 组件关联理解
- 漏洞模式抽象能力

当你能够在 IDA 中快速回答以下问题时，你已经开始进入真正的 IoT 漏洞研究阶段：

- 输入来自哪里？
- 哪些数据可控？

- 哪些路径跨越 trust boundary?
- 哪些 sink 具备危险性?
- 哪些检查可以被绕过?
- 哪些 crash 可以转化为控制流影响?

这才是 IoT 二进制漏洞研究的核心。